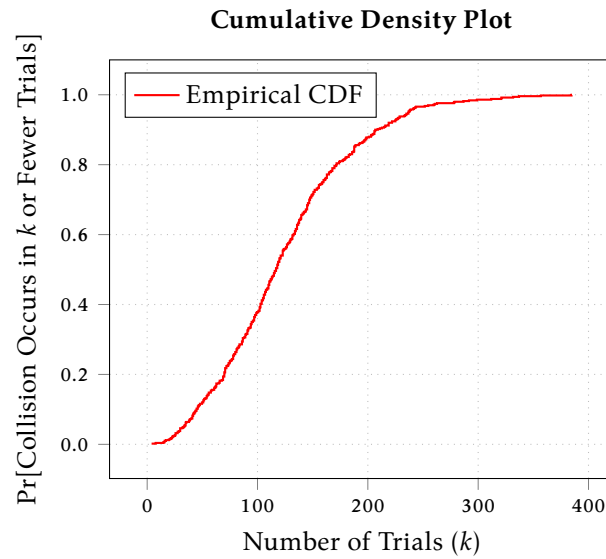
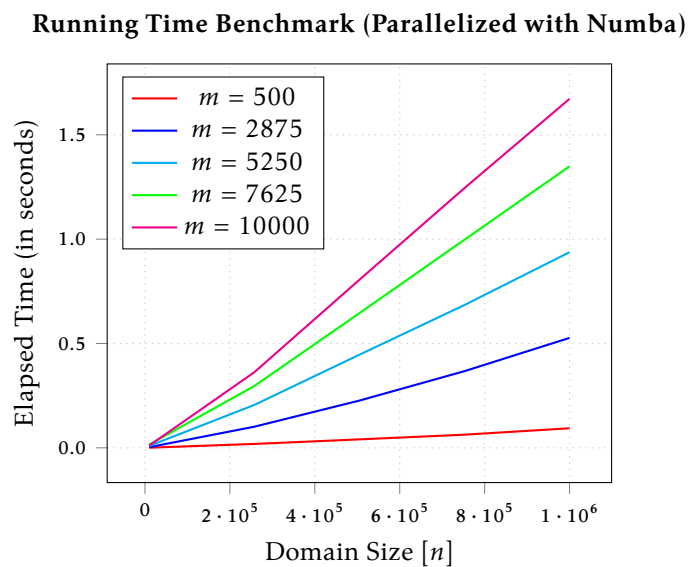


1. (A) For a domain of size 10000 a collision occurs in 120 trials.
- (B) The cumulative plot for fixed choices of $n = 10000$ and $m = 500$ resembles the following monotonic behavior:

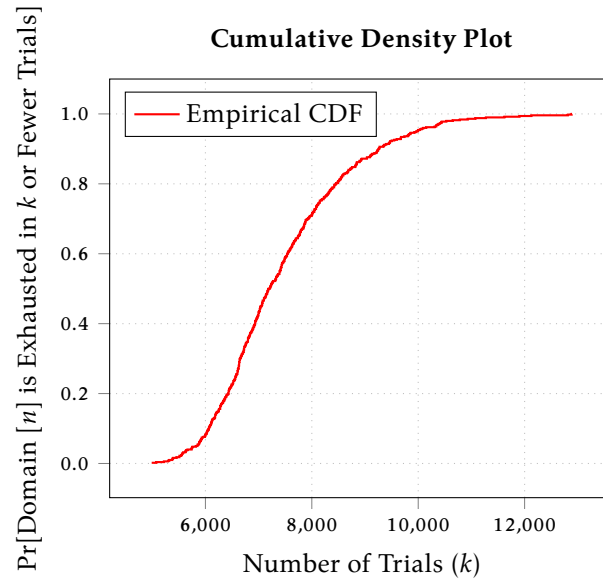


- (C) For fixed values of $n = 10000$ and $m = 500$ the expected number of trials is 122.078.
- (D) Please refer to [Listing 1](#) for implementation details. A run with $n = 10000$ and $m = 500$ took roughly 0.00066 seconds.

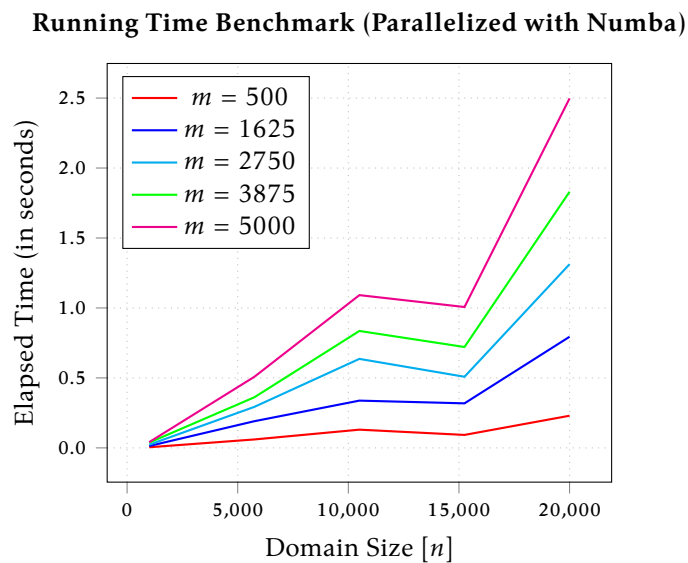


2. (A) For a domain of size 1000 all entries $i \in [1000]$ are exhausted in 7510 trials.

- (B) The cumulative plot for fixed choices of $n = 1000$ and $m = 500$ resembles the following monotonicity:



- (C) For fixed values of $n = 1000$ and $m = 500$ the expected number of trials is 7649.81.
- (D) Please refer to [Listing 2](#) for implementation details. A run with $n = 1000$ and $m = 500$ took approximately 0.0048 seconds.



3. (A) Following a similar strategic design to that of the Birthday Paradox problem from Lecture 2, we

choose $n = 10000$ to find:

$$\begin{aligned}
 1 - (1 - 1/n)^{\frac{k(k-1)}{2}} &= 1 - (1 - 1/10000)^{\frac{k(k-1)}{2}} = 1 - 0.9999^{\frac{k(k-1)}{2}} \geq 0.5 \\
 0.9999^{\frac{k(k-1)}{2}} &\leq 0.5 \\
 \frac{k(k-1)}{2} &\leq \frac{\ln(0.5)}{\ln(0.9999)} \\
 k^2 - k - \frac{2 \ln(0.5)}{\ln(0.9999)} &\leq 0
 \end{aligned}$$

Solving for $k > 0$, we find that $k = 119$, which is a numerical quantity that doesn't deviate too much from the experimental result obtained from part 1(C).

- (B) Let X denote the number of trials needed to exhaust all entries in $[n]$. Then following a similar probabilistic analysis to the Coupon Collectors problem discussed in class, we set n to 1000 to obtain (in expectation):

$$\mathbb{E}[X] = n \sum_{i=1}^n \frac{1}{i} = 1000 \sum_{i=1}^{1000} \frac{1}{i} = 7485.48.$$

This result deviates only by a small margin from the experimental result from 2(C).

4. Let f_i denote the number of trials that have value i . We can write f_i as the sum of indicator random variables $X_1 + X_2 + \dots + X_k$, where

$$X_i = \begin{cases} 1 & \text{if trial } j \text{ lands at entry } i \\ 0 & \text{otherwise.} \end{cases}$$

Since by the *union bound*, for every $\varepsilon \geq 0$ we have

$$\begin{aligned}
 \Pr \left[\left| \mu - \frac{1}{n} \right| \geq \varepsilon \right] &= 1 - \Pr \left[\left| \mu - \frac{1}{n} \right| < \varepsilon \right] \\
 &= 1 - \Pr \left[\left| \frac{f_1}{k} - \frac{1}{n} \right| < \varepsilon, \dots, \left| \frac{f_n}{k} - \frac{1}{n} \right| < \varepsilon \right] \\
 &\leq 1 - \left[1 - \sum_{i=1}^n \left(1 - \Pr \left[\left| \frac{f_i}{k} - \frac{1}{n} \right| < \varepsilon \right] \right) \right] \\
 &= \sum_{i=1}^n 1 - \Pr \left[\left| \frac{f_i}{k} - \frac{1}{n} \right| < \varepsilon \right] \\
 &= \sum_{i=1}^n \Pr \left[\left| \frac{f_i}{k} - \frac{1}{n} \right| \geq \varepsilon \right]
 \end{aligned}$$

in order to estimate a sufficiently large k that allows for all counts to be within $(\varepsilon \cdot 100)\%$ of the average with probability 0.05, it suffices to note that

$$0 \leq X_i \leq 1 \quad \text{and} \quad \frac{f_i}{k} = \frac{1}{k} \sum_{i=1}^k X_i = \bar{X} \quad \text{and} \quad \mathbb{E}[X_i] = \Pr[X_i = 1] = \frac{1}{n}$$

Invoking *Chernoff-Hoeffding Inequality* with $\Delta = 1$

$$\Pr \left[\left| \frac{f_i}{k} - \frac{1}{n} \right| \geq \varepsilon \right] = \Pr \left[|\bar{X} - \mathbb{E}[X_i]| \geq \varepsilon \right] \leq 2 \exp \left(\frac{-2\varepsilon^2 k}{1^2} \right)$$

We want a k that holds

$$\Pr \left[\left| \mu - \frac{1}{n} \right| \geq \varepsilon \right] \leq \sum_{i=1}^n 2 \exp(-2\varepsilon^2 k) \leq 0.05$$

Solving for k we obtain that $k \geq \frac{1}{\varepsilon^2} \ln(2\sqrt{10n})$.

To obtain 0.005 probability of exceeding ε error, we solve the same equation with an upperbound of 0.005 (instead of 0.05) to obtain $k \geq \frac{1}{\varepsilon^2} \ln(20\sqrt{n})$. $\square$

Appendix

```

1  import time
2  import argparse
3  import numpy as np
4
5  from numba import njit, prange
6
7  @njit('float64[:](int64, int64)', parallel=True)
8  def simulation(n: int, m: int) → np.ndarray:
9      results = np.zeros(m)
10     for i in prange(m):
11         seen = np.zeros(n)
12         count = 0
13         while count < np.inf:
14             index = np.random.randint(n)
15             count = count + 1
16             if seen[index] == 1:
17                 break
18             seen[index] = 1
19             results[i] = count
20     return results
21
22 def run_simulation(args: argparse.Namespace) → None:
23     trials = simulation(args.n, args.m)
24     print(trials)
25
26 def compute_expectation(args: argparse.Namespace) → None:
27     trials = simulation(args.n, args.m)
28     print(trials.mean())
29
30 def export_cdf_values(args: argparse.Namespace) → None:
31     trials = simulation(args.n, args.m)
32     np.savetxt('data/bp-data/trials.csv', trials, delimiter=',', fmt='%d')
33
34 def measure_duration(args: argparse.Namespace) → None:
35     tic = time.perf_counter()
36     _ = simulation(args.n, args.m)
37     toc = time.perf_counter()
38     print(toc - tic)
39
40 def run_benchmark(args: argparse.Namespace) → None:
41     ns = np.linspace(10_000, 1_000_000, args.n, dtype=int)
42     ms = np.linspace(500, 10_000, args.m, dtype=int)
43     results = np.zeros((args.n * args.m, 3))
44
45     for i, m in enumerate(ms):
46         for j, n in enumerate(ns):
47             tic = time.perf_counter()
48             _ = simulation(n, m)
49             toc = time.perf_counter()
50             results[i * args.n + j] = [n, m, toc - tic]
51
52     np.savetxt('data/bp-data/bench.csv', results, delimiter=',', header='n,m,t', comments='', fmt=['%d', '%d', '%.4f'])
53
54 def add_common_args(subparser: argparse.ArgumentParser) → None:
55     subparser.add_argument('-n', type=int, required=True, help='specify domain [n]')
56     subparser.add_argument('-m', type=int, required=True, help='specify number of trials')
57
58 def parse_args() → argparse.Namespace:
59     parser = argparse.ArgumentParser(prog='birthday-paradox', description='birthday paradox simulation')
60
61     subparsers = parser.add_subparsers(dest='command')
62
63     for cmd in ['simulate', 'expectation', 'get-cdf-vals', 'duration']:
64         add_common_args(subparsers.add_parser(cmd, description=f'runs the {cmd} command'))

```

```
65
66     benchmark_parser = subparsers.add_parser('benchmark', description='runs a benchmark')
67     benchmark_parser.add_argument('-n', type=int, required=True, help='specify number of ns')
68     benchmark_parser.add_argument('-m', type=int, required=True, help='specify number of ms')
69
70     return parser.parse_args()
71
72 def main() -> int:
73     args = parse_args()
74
75     cmd_map = {
76         'simulate': run_simulation,
77         'expectation': compute_expectation,
78         'get-cdf-vals': export_cdf_values,
79         'duration': measure_duration,
80         'benchmark': run_benchmark
81     }
82
83     cmd_map[args.command](args)
84     return 0
85
86 if __name__ == '__main__':
87     raise SystemExit(main())
```

Listing 1. birthday-paradox.py

```

1  import time
2  import argparse
3  import numpy as np
4
5  from numba import njit, prange
6
7  @njit('float64[:](int64, int64)', parallel=True)
8  def simulation(n: int, m: int) -> np.ndarray:
9      results = np.zeros(m)
10     for i in prange(m):
11         count = 0
12         unique = 0
13         seen = np.zeros(n)
14         while unique < n:
15             index = np.random.randint(n)
16             count = count + 1
17             if seen[index] == 0:
18                 unique += 1
19                 seen[index] = 1
20             results[i] = count
21     return results
22
23 def run_simulation(args: argparse.Namespace) -> None:
24     trials = simulation(args.n, args.m)
25     print(trials)
26
27 def compute_expectation(args: argparse.Namespace) -> None:
28     trials = simulation(args.n, args.m)
29     print(trials.mean())
30
31 def export_cdf_values(args: argparse.Namespace) -> None:
32     trials = simulation(args.n, args.m)
33     np.savetxt('data/cc-data/trials.csv', trials, delimiter=',', fmt='%d')
34
35 def measure_duration(args: argparse.Namespace) -> None:
36     tic = time.perf_counter()
37     _ = simulation(args.n, args.m)
38     toc = time.perf_counter()
39     print(toc - tic)
40
41 def run_benchmark(args: argparse.Namespace) -> None:
42     ns = np.linspace(1_000, 20_000, args.n, dtype=int)
43     ms = np.linspace(500, 5_000, args.m, dtype=int)
44     results = np.zeros((args.n * args.m, 3))
45
46     for i, m in enumerate(ms):
47         for j, n in enumerate(ns):
48             tic = time.perf_counter()
49             _ = simulation(n, m)
50             toc = time.perf_counter()
51             results[i * args.n + j] = [n, m, toc - tic]
52
53     np.savetxt('data/cc-data/bench.csv', results, delimiter=',', header='n,m,t', comments='', fmt=['%d', '%d', '%.4f'])
54
55 def add_common_args(subparser: argparse.ArgumentParser) -> None:
56     subparser.add_argument('-n', type=int, required=True, help='specify domain [n]')
57     subparser.add_argument('-m', type=int, required=True, help='specify number of trials')
58
59 def parse_args() -> argparse.Namespace:
60     parser = argparse.ArgumentParser(prog='coupon-collector', description='coupon collector simulation')
61
62     subparsers = parser.add_subparsers(dest='command')
63
64     for cmd in ['simulate', 'expectation', 'get-cdf-vals', 'duration']:
65         add_common_args(subparsers.add_parser(cmd, description=f'runs {cmd} command'))
66

```

```
67     benchmark_parser = subparsers.add_parser('benchmark', description='runs a benchmark')
68     benchmark_parser.add_argument('-n', type=int, required=True, help='specify number of ns')
69     benchmark_parser.add_argument('-m', type=int, required=True, help='specify number of ms')
70
71     return parser.parse_args()
72
73 def main() -> int:
74     args = parse_args()
75
76     cmd_map = {
77         'simulate': run_simulation,
78         'expectation': compute_expectation,
79         'get-cdf-vals': export_cdf_values,
80         'duration': measure_duration,
81         'benchmark': run_benchmark
82     }
83
84     cmd_map[args.command](args)
85     return 0
86
87 if __name__ == '__main__':
88     raise SystemExit(main())
```

Listing 2. coupon-collector.py